

이제부터 할 일 — 프로토타입 기획 (동작 가능 vertical slice)

선행: 이벤트 시스템 구현 - 안 4 (kind 분리 + chain).md , 이벤트 시스템 - 1B 검증 결과.md

목적: 5천자 내외 시나리오 + 분기 1개 이상 + 한 회차 완주 가능한 동작 프로토타입까지의 세부 기획. 시스템·컨텐츠·검증을 단계별로 묶음.

§0. 프로토타입의 정의 (PASS 조건)

플레이어가 게임을 켜고 다음 흐름이 끊김 없이 진행됨:

1. **run_start** — 시작 대사 발화.
2. **맵 진행** — event 노드 1개 이상 진입 → 콘텐츠 트리거.
3. **분기 1회 이상** — 선택지 한 곳에서 의미 있는 결정.
4. **결과 차이 체감** — 선택에 따라 능력치 / 이후 라인 / 분기 다름.
5. **회귀** — research 노드 → 회귀 대사 발화.
6. **2회차 이상 진행 가능** — 첫 회차에서 본 이벤트는 once_per 로 필터.

부수 조건: - **시나리오 분량**: 5천자 (~125 라인, 이벤트 ~15~25개). - **자산**: 현재 있는 것 (히로인 idle, 맵, 전투 UI) 외 추가 0. - **시각 연출**: 대사창 + 선택지 버튼만. portrait / 배경 / 영상 0. - **회귀 자동 검증**: 1A·1A-2·1B 65 체크 + 신규 단계 누적.

§1. 시스템 — 3 단계 누적

1-1. Stage 2A — effect kind

산출물 - event_manager.gd::begin_event 의 match kind 에 "effect" 추가. -

_dispatch_effect(event_id, def) — event_state = {kind: "effect", phase: "running", chain_root_id, event_id} 설정 → _apply_event_actions(def.effects) →

_finalize_event() . - _apply_event_actions(actions: Array) — 각 action 의 type 보고 RunManager._apply_<type>(params) 호출. 모르는 type 은 push_warning + 건너뛸.

스키마 예

```
"data_pickup": {
  "id": "data_pickup",
  "kind": "effect",
```

```

    "trigger": {...},
    "effects": [
      {"type": "research_data_add", "amount": 10},
    ],
  }

```

필요한 RunManager 신규 함수 - `_apply_research_data_add(params)` —

`big_run_data["research_data"] += params.amount` . - (이미 있는 `_apply_body_boost` / `_apply_arm_attack_boost` / `_apply_arm_durability_boost` 는 effect kind 에서도 그대로 재사용.)

검증 시나리오 - effect 노드 진입 → 즉시 `_apply_*` 호출 → `seen_events` 증가 + `phase = "map"` 자동 복귀. UI 입력 없음. - 검증 항목 ~10개 (단계 1A 의 절반 정도).

예상 작업 분량: 코드 ~50줄 + 테스트 ~80줄.

1-2. Stage 2B — chain (`next` 필드)

산출물 - EventDefinition 에 옵션 필드 `next: String` . - `_finalize_event` 직전 분기: - 현재 def 의 `next` 가 있고 `EVENTS[next]` 존재 → `event_state.event_id = next` 갱신 + kind 별 재초기화 (`line_idx = 0` 등) + `event_state_changed.emit()` . **`event_resolved` emit 안 함, phase 안 빠짐.** - 없거나 chain 끝 → 기존 `_finalize_event` 그대로. - `chain_root_id` 가 chain 의 첫 `begin_event` 에 결정, 이후 노드 전이 시 그대로 유지. - `seen_events` 카운트 = `chain_root_id` 만.

스키마 예

```

"first_intro": {
  "id": "first_intro",
  "kind": "dialogue",
  "trigger": {"type": "run_start"},
  "once_per": "big_run",
  "lines": [{"speaker": "히로인", "text": "..."}],
  "next": "first_intro_effect",
},
"first_intro_effect": {
  "id": "first_intro_effect",
  "kind": "effect",
  "effects": [{"type": "body_boost", "amount": -10}],
  "next": "first_intro_aftermath",
},
"first_intro_aftermath": {
  "id": "first_intro_aftermath",
  "kind": "dialogue",
  "lines": [{"speaker": "히로인", "text": "..."}],
}

```

검증 시나리오 - chain 진행 동안 `phase == "event"` 유지. - `chain_root_id` 가 chain 진행 내내 고정. - `seen_events` 가 root 한 개만 카운트. - 잘못된 `next` (EVENTS 미등록) → `push_warning` + 정상 종료.

예상 작업 분량: 코드 ~30줄 + 테스트 ~100줄.

1-3. Stage 2D — `choice` kind

산출물 - `event_manager.gd::begin_event` 분기에 `"choice"` . - `_dispatch_choice(event_id, def)` — `event_state = {kind: "choice", phase: "awaiting_input", prompt: def.prompt, choices: def.choices, ...}` . - 신규 메서드 `select_choice(idx: int)` — 유효성 검사 → `event_state.event_id = choices[idx].next` → kind 재초기화 → `event_state_changed.emit()` . - `event_ui.gd` 에 choice 분기 추가 — prompt 라인 + 버튼 N개 렌더, 클릭 → `select_choice(idx)` . - LimboConsole: `event_choose <idx>` .

스키마 예

```
"crossroads": {
  "id": "crossroads",
  "kind": "choice",
  "trigger": {...},
  "prompt": {"speaker": "히로인", "text": "어떻게 할까?"},
  "choices": [
    {"label": "강행", "next": "crossroads_push"},
    {"label": "후퇴", "next": "crossroads_retreat"},
  ],
}
```

검증 시나리오 - prompt 표시 → 두 가지 → 가지 A 선택 시 그 `next` 로 / B 선택 시 다른 `next` 로 chain 분기. - chain root 는 choice 이벤트의 id 로 고정. - 헤드리스 콘솔 (`select_choice` 직호출) + 실 플레이 (스크린샷 1~2 컷).

예상 작업 분량: 코드 ~80줄 (UI 포함) + 테스트 ~120줄.

§2. 콘텐츠 — 시나리오 풀 (5천자)

대략 ~15~20개 이벤트 / 라인 ~125개. 메모리 — 히로인 이중 정체성 아크 의 초반 병기성 결.

시나리오 흐름 (제안)

단계	트리거	이벤트 ID	비고
1차 시작	run_start	arc_intro_chain	dialogue → effect (현황 진단) → dialogue. chain 사용.

단계	트리거	이벤트 ID	비고
첫 노드 진입	node_enter type "event"	early_observation	짧은 모놀로그.
분기 노드	node_enter type "event_choice"	arms_assessment_choice	choice — "강행 / 후퇴" 같은 결정. 결과로 능력치 변동.
회귀 직전	node_enter type "research" 직전 또는 사이드 노드	pre_regression_chain	dialogue → effect → dialogue.
회귀 직후	run_start (2회차+)	recurrence_remark	once_per 또는 단순 일관 트리거.

(맵 노드 type 은 일부 신설 — event_choice / event_pre_regression 등. 또는 모두 event 로 두고 once_per + 가중치로 분배.)

라인 가이드라인

- 라인당 ~30~50자.
- 한 이벤트 5~10 라인.
- 화자 = "히로인" 단일 (CHARACTERS 도입 전).
- 메모리 — 초기엔 자기 자신을 병기로만 인식. "팔" / "기능" / "출력" 같은 용어 위주.

§3. 통합 검증 (프로토타입 PASS)

3-1. 헤드리스 (자동 회귀)

기존 65 체크 + 신규: - 2A: ~10 체크 - 2B: ~15 체크 - 2D: ~15 체크 - 누적 ~105 체크.

매 단계 진입 시 전체 회귀 PASS 확인.

3-2. 실 플레이 (1회 수동)

체크리스트: - [] 시작 → intro chain 끝까지 자동 진행 - [] 맵 노드 클릭 → event 노드 진입 시 대사창 노출 - [] choice 노드에서 두 선택지 모두 시도 (1회씩) — 결과 차이 확인 - [] research 노드 진입 → 회귀 대사 → 새 회차 시작 - [] 2회차 시작 시 1회차 once_per 이벤트 미발화 확인 - [] 한 회차 완주 시간 ~10분 이내

3-3. 콘텐츠 검증 (선택)

- 시작 시 1회 _validate_events() 호출 — next 깨짐 / kind 오타 / 빈 lines 검출.
- 콘솔 에러 0 = PASS.

§4. 진입 순서·예상 분량

단계	예상 시간 (코드+검증)	의존
Stage 2A — effect	~1시간	없음
Stage 2B — chain	~1.5시간	2A
Stage 2D — choice	~2시간	2B
컨텐츠 작성 (5천자)	작가 시간	2D
통합 검증	~30분	컨텐츠
(옵션) <code>_validate_events</code>	~30분	컨텐츠

(이전 단계 1A 가 ~30분 + 1A-2 ~30분 + 1B ~1시간 정도 걸림. 위 예상은 대략적.)

§5. 의도적 제외 (프로토타입 단계)

제외 항목	이유
portraits / backgrounds / movies	자산 의존. 프로토타입은 텍스트만.
CHARACTERS 레지스트리	speaker freeform String 으로 충분. 화자 1명.
변수 인터폴레이션 (<code>{run.body_hp}</code>)	5천자 내수용. 동적 텍스트 필요한 건 1~2 라인 정도라 그냥 effect 후 다른 라인 으로 처리.
외부 <code>.json</code>	5천자 = const dict 충분. 임계점 미달.
<code>set_flag</code> / 플래그 조건 트리거	회귀 1~2회 분량 = 플래그 없이도 <code>once_per</code> + <code>chain</code> 으로 표현 가능.
트리거 확장 (combat_victory 등)	프로토타입은 <code>run_start</code> + <code>node_enter</code> 만으로 충분. 전투 후 대사는 사이드 노드 로 우회.
<code>once_per: "internal_run"</code>	회귀마다 발화 필요한 컨텐츠가 들어오면 추가. 프로토타입은 <code>once_per:</code> <code>big_run</code> 으로 통일 가능.

위 항목들은 프로토타입 검증 후 다음 차로. 컨텐츠 양이 늘어나거나 작가 워크플로 변할 때 자연스럽게 도입.

§6. 핵심 결정 포인트 (사전 합의)

진입 전에 정해둘 한두 가지:

6-1. 맵 구조 — event 노드 몇 개?

- 현재 노드 5 한 개만 type "event". 노드 4 는 research.
- 프로토타입 = event 노드 2~3 개로 늘려야 콘텐츠 풀 활용 가능.
- 결정 필요: TEST_MAP_GRAPH 의 어떤 노드를 event 로 추가할지. 또는 type 다양화 (event_intro / event_combat / event_choice).

6-2. 회차 분량

- 한 회차 동안 event 노드 몇 개 거치게? 2~4 개 적당.
- 회귀 1회로 끝낼지, 2~3회 회귀 후 종결인지.
- 결정 필요: 프로토타입 PASS 의 "한 회차 완주" 정의 (회차 수, 노드 통과 수).

6-3. 분기의 의미

- choice 의 결과 차이가 ① 능력치 (effect) ② 이후 라인 (chain 분기) ③ 둘 다.
- 가장 단순 = ③. choice → next 가 다른 chain → 각 chain 안에 다른 effect.
- 결정 필요: 분기 1 곳의 구체 형태 ("강행 +HP / 후퇴 +data" 식).

§7. 즉시 다음 액션

Stage 2A 진입 GO 신호 + §6 의 3 결정 받으면 코드 + 헤드리스 검증 + PDF 흐름 그대로 진행.

§6 결정은 콘텐츠 작성 시점까지 연기 가능 — 시스템 단계 (2A·2B·2D) 는 결정 무관하게 진입 가능.